Member 5 — Analytics, Safety and Quality

Member placeholder: [MEMBER 5 NAME]

Proposed viva responsibility: temperature-safety policy, actual-versus-clinical-versus-core semantics, seven-day personal baseline, deterministic alert engine, and the test/audit evidence classification used by the team.

Authorship disclaimer: this document maps the repository's analytics/safety/quality capabilities to a proposed HOD responsibility. It does not prove that [MEMBER 5 NAME] authored these files. The team must confirm authorship; use **"the team implemented"** for source-backed shared work.

A. Responsibility scope

A.1 One-sentence scope

For this discussion, [MEMBER 5 NAME] is responsible for explaining how the project prevents a sensor reading from becoming a false clinical claim — through exact temperature semantics, a clearly non-diagnostic baseline, deterministic rules, and evidence labels that separate source, tests, builds, prior observations and future plans.

A.2 Included in this responsibility

- temperature_safety.py: actual TMP117 versus clinical wrist versus core; signed raw conversion/tolerance; stuck-pattern and range rules; unverified/verified wrist paths; exact fever message and unavailable policy.
- personal_baseline.py: one active enrollment per device/wearer; seven days and at least 30 sampled minutes per day; minute-mean aggregation; median/MAD frozen reference and engineering noise floors; deviation events after three consecutive outside readings; why unverified wrist data blocks an overall temperature model.
- ai_decision_engine.py: deterministic SOS/fall/HR/SpO2 status; validated-only extraction; unavailable values not substituted; actual TMP117 reported separately; terminal alert card; no trained model, no LLM.
- Quality evidence: tests and builds; audit/cleanup/next-step records; what is software-resolved versus hardware/clinical-verification-required; honest limitations and release gates.

- Cross-module explanation of how the same safety policy is enforced in firmware, bridge, store, dashboard and terminal output.

A.3 Outside primary scope — but required knowledge

- Firmware scheduling/integration: Member 1.
- Physical sensors/wiring/diagnostics: Member 2.
- Serial/HTTP/SSE backend: Member 3.
- Dashboard/storage rendering: Member 4.

Member 5 must still trace the policy into all those layers; the safety model is not a standalone opinion layer.

A.4 Key files

File	Role
temperature_safety.py	Shared temperature/fever assessment and exact message
personal_baseline.py	Seven-day per-wearer statistical baseline and deviation events
ai_decision_engine.py	Deterministic clinical summary/terminal alert card
telemetry_store.py	Clinical wrist acceptance and per-wearer observations
tests/test_temperature_safety.py	Temperature/fever boundary tests
tests/test_baseline.py	Baseline qualification/freeze/deviation tests
tests/test_decision_engine.py	Deterministic engine tests
CODE_AUDIT_MANIFEST.md	Evidence inventory, prior observations and readiness statement
CODE_CLEANUP_REGISTER.md	Software-closed findings and open verification items
NEXT_STEPS.md	Ordered hardware/contact/release work

B. Achievements attributable to this area

Achievement	What the repository	Evidence

	contains	
Exact temperature semantics	Actual TMP117, clinical wrist, core and fever are distinct	[SOURCE] temperature_safety.py; [TEST] Python/Node
Fail-closed temperature assessment	Invalid raw/value/timestamp/range or mismatch → sensor_error and null evidence	[SOURCE]; [TEST]
Unverified contact cannot expose a wrist value	Current packet always unverified, null value/raw/timestamp	[SOURCE]; [TEST] across all layers
No core/fever invention	Core null/not_available; fever not_available with exact message	[SOURCE]; [TEST]
Seven-day baseline	Per-wearer enrollment, 7 days, 30 minutes/day, minute means, frozen median/MAD	[SOURCE]; [TEST] baseline tests
Conservative deviation events	Three consecutive outside readings within 5 s, 300 s cooldown	[SOURCE]; [TEST]
Deterministic alert summary	Validated-only fields, SOS/fall/HR/SpO2 rules, explicit unavailable values	[SOURCE]; [TEST]
No false AI claim	Module is legacy-named; no trained model/LLM	[SOURCE] docstring/code; documentation review
Software quality record	Closed findings C-01–C-14; open hardware verification O-01–O-07	[SOURCE] audit/register/next steps
Reproducible checks	62 Python + 10 Node tests; Ruff/compile/syntax clean; both builds	[TEST]/[BUILD] fresh 2026-10-29

B.1 What this area does not establish

- That the sensors are physically accurate.
- That wrist contact can be verified.
- That core temperature can be estimated or fever diagnosed.

- That the baseline is clinically validated or diagnostic.
- That the alert rules are clinically appropriate for real patients.
- That the hardware/end-to-end device works.
- That SHT40/MLX90614 are installed or usable.
- That the project is regulatory/clinical-ready.

C. Detailed code walkthrough

C.1 Temperature policy — temperature_safety.py

Constants:

Constant	Value	Meaning
TMP117_MIN_C / TMP117_MAX_C	-55 / 150	Accepted actual-sensor code range
WRIST_MIN_C / WRIST_MAX_C	-10 / 60	Clinical wrist range only with verified contact
RAW_LSB_C	0.0078125	°C per signed LSB
RAW_TOLERANCE_C	0.02	Allowed difference between reported value and signed conversion
FEVER_MESSAGE	**Core temperature unavailable — confirmation required**	Exact user-facing message
CORE_STATUS	not_available	Core-temperature status
UNCERTAINTY	"No validated core-temperature estimate or core reference is available."	Uncertainty statement
LIMITATIONS	Wrist is affected by ambient conditions, blood flow, contact pressure and activity; wrist alone cannot confirm/rule out fever	Limitation statement

evaluate_temperature_safety(packet):

1. Reads only dictionaries; malformed/non-dict input becomes empty.

2. Validates status strings against allowed sets; `unverified` is not allowed for the actual sensor.

3. Masks actual sensor fields if `sensors.tmp117` is not true.

4. If actual status is `valid`, requires finite value, usable raw (not `0x0000`/`0xFFFF`), timestamp, value in `-55..150`, signed conversion present, and absolute conversion error ≤ `0.02 °C`. Any failure becomes `sensor_error` with value/raw/timestamp null.

5. Non-valid actual statuses publish null value/raw/timestamp.

6. Contact status defaults to `unverified` if unrecognized.

7. `unverified` forces wrist status `unverified` and null wrist value/raw/timestamp.

8. `verified` requires all matching evidence: valid actual, valid wrist, usable raw, timestamps, actual raw == wrist raw, actual timestamp == wrist timestamp, wrist in `-10..60`, actual/wrist numeric agreement within `0.02`, and raw conversion within `0.02`. Any failure becomes `sensor_error` and nulls the wrist evidence.

9. Builds fever input with measurement `wrist_skin_temperature`, unit `C`, quality, `measured`, timestamp/raw only when a wrist value exists.

10. Returns fever `status: not_available`, `screening_supported: false`, exact message, `rule: None`, uncertainty/limitations.

11. Always returns core null/`not_available`.

This function is the central explanation for the whole project: **a TMP117 package reading is not automatically a clinical wrist reading, and neither is a core reading.**

C.2 Personal baseline — `personal_baseline.py`

The module docstring is explicit: **"An explicit seven-day, per-wearer statistical reference (not a diagnostic model)."**

Enrollment

- `start(device_id, person)` requires a non-empty person/alias up to 80 characters and a device ID up to 128 characters.
- It refuses if the device already has an active enrollment; end it before changing wearer.
- It creates `enrollments` with a UUID, device, person, start time, active flag and empty model JSON, and inserts a `collection` event.

- A second wearer never inherits the previous model set.
- `end(device_id)` deactivates the enrollment and clears in-memory comparison runs for that device.

Qualification

- Constants: `REQUIRED_DAYS = 7`, `MIN_MINUTES_PER_DAY = 30`.
- `summary()` caches results for 30 seconds unless `force=True`.
- Observations are reduced to minute means using `AVG(...)` grouped by `CAST((recorded_at - started)/60 AS INTEGER)`.
- For each active metric, only non-null minute means count.
- `counts[day]` counts distinct sampled minutes with a non-null mean; a day qualifies at ≥30 such minutes.
- `qualified >= 7` is required.
- `elapsed` also requires at least seven days since the first non-null minute, not just seven calendar labels.
- Only then is a model created.

Model

`center = median(values)`

`width = max(3 × 1.4826 × MAD, noise floor)`

`low = center - width`

`high = center + width`

Noise floors (engineering, not clinical):

Metric	**Floor**
Heart rate	5.0 bpm
SpO2	2.0 %
Wrist skin temperature	0.2 °C
Motion	0.1 g

Additional clamps: SpO2 range `0..100`; motion lower bound `0`. The model stores `center`, `low`,

high, mad, creation time, minutes, qualified days and method text. It is **frozen** once created; later values do not retrain it.

Readiness

- Per-metric status: ready if a model exists, collecting if values exist, waiting otherwise.
- Overall status: ready only if all four active metrics have models; partial if some do; collecting if none do.
- Because wrist contact is unverified and the store does not retain a wrist value, the wrist model cannot qualify. Therefore overall baseline readiness cannot be reached in the current build.
- This is intentional: the project documents the limitation instead of injecting the actual TMP117 value into the wrist column.

Comparison and events

compare(packet):

- calls measurements(packet) (valid values only; unverified wrist is null);
- for each active metric: no value → unavailable; no model → collecting; outside low..high → outside; otherwise within;
- a run increments only when state is outside and the prior reading was within 5 s; otherwise it resets/starts at 1;
- at count ≥ 3 and at least 300 s since the last event for that metric, it inserts a deviation event with a human-readable detail;
- events() returns at most 30 events;
- run state is in memory; a process restart resets the consecutive count. This is not persisted as a clinical episode.

C.3 Deterministic alert engine — ai_decision_engine.py

Module name: legacy "AI" naming; the code contains deterministic rules only. There is no trained parameter set, neural network, classifier or LLM call.

_measurement(value) returns a finite number or None. _validated(vitals, key, status_key, sensor, sensors) requires the sensor boolean true, status exactly valid, and a finite measurement.

evaluate_clinical_status(packet):

1. Requires sensors.max true and finger_on true for HR/SpO2; otherwise both are None even if old values exist.
2. Validates motion through _validated with mpu.
3. Calls evaluate_temperature_safety() for actual/wrist/core/fever.
4. Only reads legacy MLX/SHT fields if the corresponding sensor boolean is true; both are false in the current build.
5. Status logic:
   - ○ SOS or fall → **RED (EMERGENCY)**; action "Check on the person immediately."
   - ○ Else HR < 50 or > 120, or SpO2 < 90 → **YELLOW (WARNING)**; check placement and notify caregiver if persistent.
   - ○ Else both HR and SpO2 valid → **GREEN (NO OPTICAL ALERT)**.
   - ○ If either optical value is missing, the status becomes **UNKNOWN (MISSING DATA)** even when no threshold was crossed.
6. Adds explicit findings for unavailable optical values; no zeros substituted.
7. Adds actual TMP117 finding: actual sensor, not clinical wrist/core.
8. Adds wrist finding: wrist context, not core; current status unverified/null.
9. Always appends the exact fever message and the explanation that no core method is validated.
10. Always appends: confirm suspected fever with a validated clinical thermometer and clinician assessment; do not diagnose from wrist skin.
11. Hardware health counts only installed sensors; OLED is shown as display online/offline.
12. Returns patient_metrics, full temperature, findings/actions and status_summary.

Important output semantics: estimated_core_temperature_c is always None; estimated_core_temperature_status is not_available; fever_screening_status is not_available; fever_screening_message is the exact message; actual TMP117 and wrist use UNAVAILABLE strings in the terminal metric dictionary when null, without invented numbers.

C.4 Terminal alert card

print_alert_card() prints a bounded-width card with timestamp/status, hardware health, HR/SpO2, actual TMP117 ("actual sensor, not core"), wrist skin ("not core"), core "not available," the exact fever message, optional legacy lines only if present (they are not), steps/motion, findings and caregiver recommendations. This is a terminal prototype output, not a medical device display.

D. Technical decisions and trade-offs

Decision	Reason	Trade-off
Actual TMP117 separate from clinical wrist	Package temperature is not a wrist measurement and cannot be promoted	Clinical metric unavailable until contact is verified
Clinical wrist separate from core	Wrist is peripheral context; no validated core transformation exists	No core number presented
No fixed offset	Ambient/contact/perfusion effects make a constant offset indefensible	Absolute temperature interpretation remains limited
Stuck raw and range/mismatch rejection	Prevents fake near-zero or stale values	Some genuinely extreme readings require investigation rather than display
Exact fever message with em dash	Consistent user-facing clinical boundary across layers	Font/terminal/OLED rendering must be checked; OLED uses compact text
Core/fever always unavailable	Avoid implying a validated capability	Dashboard is less "complete," but honest
Seven-day, 30-minute qualification	Reduces noise and avoids instant "baselines"	Slow; wrist prevents overall readiness in current build
Median/MAD with noise floors	Robust to outliers; minimum practical width	Not a clinical reference interval; not validated
Frozen model	Stable comparison; no continuous self-reinvention	Cannot adapt to a genuine long-term change without a new enrollment
Three readings/300 s cooldown	Avoid event spam from transient excursions	Prototype heuristic; false positives/negatives possible

Decision	Reason	Trade-off
No threshold alert on actual TMP117 or wrist	No validated clinical threshold exists	Some users may expect a temperature alert; the project refuses to invent one
Deterministic engine instead of ML	Explainable, testable, no training-data claim	Not adaptive; thresholds are engineering rules
"AI" name retained for historical compatibility	Avoid unnecessary rename churn	Must repeatedly explain that no AI model exists
Audit evidence tags	Prevent tests/builds from being confused with hardware/clinical proof	Requires discipline in every viva answer

D.1 Why the baseline is not a diagnosis

A statistical departure means "this value differs from this wearer's recorded reference." It does not mean disease, fever, hypoxia or a clinical event. The model is built from unverified sensor data and engineering noise floors; it has no clinical validation. The defensible statement is: **the value stayed outside the stored personal reference for three readings and an event was recorded for review.**

D.2 Why the alert engine is not AI

It has explicit `if/elif` thresholds and string status labels. There is no learned weight, no inference model, no training dataset and no LLM call. The word "AI" in the filename is historical; the code docstring describes a threshold alert engine. Never present the module name as evidence of AI capability.

E. Cross-module contributions and interfaces

E.1 Safety policy → firmware/bridge/dashboard/store

`temperature_safety.py` is mirrored/re-enforced across layers:

Layer	Enforcement
Firmware	Actual TMP117 fields separate; wrist always unverified/null; core null; fever exact/unavailable

Layer	Enforcement
Bridge	`_validate_temperature()` and `_validate_fever()` reject contradictions
Store	`measurements()` stores wrist only with verified contact and matching evidence
Dashboard	`metricStatus()`/`temperatureAssessment()` mask and label; exact labels are **Wrist Skin Temperature (°C)** and **TMP117 Sensor Temperature (°C)**; core/fever unavailable
Terminal engine	Reports actual separately; no core/fever invention

This redundancy is intentional. A single missed safety check should not be enough to expose a false clinical value.

E.2 Baseline → storage/dashboard

- `PersonalBaseline` reads `TelemetryStore.measurements()` and observations; it does not have a private data source.
- The dashboard consumes `/api/baseline` and `/api/events`; it shows model presence/ranges and comparison state.
- Unverified wrist data never reaches the baseline because the store refuses to persist it.
- A new wearer gets an empty model set and a new enrollment; old data stays associated with its device/enrollment.

E.3 Engine → UI/serial

The terminal engine can consume a packet directly or listen to serial via `telemetry_bridge.parse_packet`. It calls the same temperature policy. It does not publish a separate clinical core/fever result.

E.4 Quality evidence → team claims

1. Say `[SOURCE]` only for implemented behavior.
2. Say `[TEST]` only for executed host tests, with date/count.
3. Say `[BUILD]` only for compiled/linked targets.

4. Say [RECORDED HW OBS] for prior hardware observations, explicitly not reproduced.

5. Say [UNVERIFIED HW] for source/build that has not been physically validated.

6. Say [PLAN] for future work.

This prevents the most common viva failure: converting "the code supports it" into "the device does it."

F. Evidence classification for this area

Claim	Classification	Verification
Temperature safety separates actual/wrist/core/fever	[SOURCE] + [TEST]	temperature_safety.py; Python/Node tests
Actual range -55..150; wrist -10..60 only with verified contact	[SOURCE] + [TEST]	constants, tests
Raw 0x0000/0xFFFF rejected	[SOURCE] + [TEST]	safety/bridge/dashboard, vector tests
Core null/fever exact message	[SOURCE] + [TEST]	safety/engine/bridge/dashboard
Baseline 7 days/30 minutes/minute means/MAD	[SOURCE] + [TEST]	personal_baseline.py; baseline tests
Deviation after 3 readings/300 s	[SOURCE] + [TEST]	compare tests
Engine deterministic thresholds	[SOURCE] + [TEST]	ai_decision_engine.py; engine tests
No trained model/LLM	[SOURCE]	code/docstring; no model artifacts/dependencies
Audit register C-01–C-14 closed in software	[SOURCE]	CODE_CLEANUP_REGISTER.md
O-01–O-07 open hardware/validation/CI items	[SOURCE]	register/next steps
62 Python + 10 Node tests pass	[TEST] fresh 2026-10-29	commands
Both builds succeed	[BUILD] fresh 2026-10-29	PlatformIO output
Sensor accuracy/contact/core/fever	[UNVERIFIED HW]/not established	no reference/clinical study

Claim	Classification	Verification
clinical validity		
Baseline clinical validity	**Not established**	statistical engineering prototype only

F.1 Audit record details to use carefully

CODE_CLEANUP_REGISTER.md says "resolved" means **the listed software behavior is covered by source review and host tests/builds; it does not mean physical hardware or clinical use was validated.**

Closed software findings include C-01 actual TMP117 versus wrist conflation; C-02 unverified contact exposure; C-03 clinical range wrongly applied to actual sensor; C-04 raw conversion/stuck patterns; C-05 stale/inconsistent evidence; C-06 duplicate JSON keys; C-07 implied core/fever from wrist; C-08 legacy aliases/optional sensor counting; C-09 OLED line bounds; C-10 missing-history continuity/zero; C-11 actual sensor as clinical history; C-12 baseline from unverified wrist; C-13 label clarity; C-14 test coverage.

Open verification items include O-01 TMP117 physical address/identity/config/raw, O-02 accuracy, O-03 contact method, O-04 MAX/MPU behavior, O-05 OLED/SOS/end-to-end, O-06 clinical/regulatory suitability, O-07 CI automation.

Do not say "all audit items are closed." The software findings are closed; the hardware/validation items are open.

F.2 Fresh evidence

- python -m pytest -q → **62 passed**.
- node --test tests/dashboard_core.test.js → **10 passed**.
- Ruff/Python compile/JS syntax → passed.
- Main firmware → SUCCESS, RAM 21,424/327,680, flash 326,001/6,553,600.
- TMP117 diagnostic → SUCCESS, RAM 18,916/327,680, flash 282,377/6,553,600.
- No fresh hardware upload/capture/reference measurement/contact test/end-to-end assembled-device test.

G. Debugging: safety/analytics scenarios

G.1 “The TMP117 shows 29.65 °C but the wrist card is blank”

This is correct. `29.65 °C` is actual sensor temperature; wrist requires verified contact and matching wrist evidence. `evaluate_temperature_safety()` forces wrist null/unverified. Do not copy actual into wrist or add an offset.

G.2 “The dashboard/terminal says core unavailable”

That is the designed output. There is no validated core method. Confirm suspected fever with a validated clinical thermometer and clinician assessment; do not infer fever from wrist or actual sensor temperature.

G.3 “The baseline never becomes fully ready”

Check person/enrollment, seven elapsed days since the first non-null minute, at least 30 sampled minutes per qualifying day, HR/SpO2/motion models, and wrist unverified/null. The overall `ready` state requires all four active metrics. The wrist blocker is intentional; injecting actual TMP117 into the wrist column would be a safety regression.

G.4 “A deviation event fired but the person feels fine”

The event means three consecutive readings were outside the stored personal reference. It is not a diagnosis. Check sensor placement, signal quality, activity and the frozen model range. Avoid claiming clinical significance.

G.5 “The engine says UNKNOWN even though there is no threshold alert”

If HR or SpO2 is unavailable, the engine intentionally returns `UNKNOWN (MISSING DATA)` rather than `GREEN`. Green requires both optical values valid; absence is not reassurance.

G.6 “The engine is called AI, so where is the model?”

There is no model. The file name is historical; the implementation uses deterministic thresholds. Do not invent training data, weights or an LLM.

G.7 “A prior TMP117 read gave -0.01 °C”

Treat as a prior observation/hypothesis, not a current diagnosis. Raw `0xFFFF` converts to `-0.0078125`, which rounds to `-0.01`; the diagnostic must confirm all-ones and the electrical

cause. No offset should be added.

G.8 "A test passes, so the feature is safe"

A host test proves a code path on synthetic fixtures. It does not prove sensor accuracy, contact, clinical validity, real serial timing or emergency reliability. Re-state the evidence tag.

H. Testing and verification

Test area	What it covers
test_temperature_safety.py	Actual/wrist/core/fever, raw/range/stuck evidence, verified/unverified, exact message
test_baseline.py	Enrollment/qualification/freeze/deviation/noise floors/wrist blocker
test_decision_engine.py	SOS/fall/HR/SpO2 statuses, missing optical data, actual/wrist/core/fever output
test_telemetry.py	Schema/status/raw/fever validation, duplicates, stale replay
test_history.py	Valid-only storage, unverified wrist rejection, gaps, device isolation
dashboard_core.test.js	Pure dashboard temperature/status/gap/CSV behavior
test_tmp117_vectors.py	Signed conversion/stuck/range vectors

Fresh checks: 62 Python passed; 10 Node passed; Ruff/compile/JS syntax passed; both PlatformIO builds succeeded with the RAM/flash figures above.

What the tests do not prove: sensor accuracy or contact; clinical validity of thresholds/baseline; real hardware timing, OLED/SOS/fall behavior; long-duration reliability, load, security, disk failure; regulatory suitability.

Quality gate:

1. Run all host tests and syntax/lint checks.
2. Rebuild both firmware targets.
3. If source changed, rerun and date the results.
4. Run the TMP117 raw diagnostic on hardware for ≥5 minutes.

5. Complete reference/contact/MAX/MPU/OLED/SOS/end-to-end checks.

6. Update evidence labels; keep prior observations labeled not reproduced.

7. Do not claim clinical/regulatory readiness without the appropriate external process.

I. HOD preparation

I.1 60–90 second explanation

"My allocated area is analytics, safety and quality. The central safety rule is that the actual TMP117 sensor temperature, the clinical wrist-skin temperature, core temperature and fever screening are separate. The actual sensor is range-checked to minus 55 to 150 degrees; the wrist range is minus 10 to 60 and only applies with verified contact. The current build always has unverified contact, so the wrist value, raw and timestamp are null; core is null and fever is not available with the exact message that core temperature is unavailable and confirmation is required. No fixed offset is used. The baseline is a seven-day, per-wearer median and MAD reference requiring thirty sampled minutes per day; it freezes once built and records a deviation only after three consecutive outside readings. The alert engine is deterministic: SOS or fall, heart-rate and SpO2 thresholds, otherwise unknown when optical data is missing. It is not a trained model or an LLM. The sixty-two Python and ten Node tests pass and both builds succeed, but sensor accuracy, contact, clinical validity and the assembled device remain unverified."

I.2 Technical questions and answers

1. **What are the four temperature concepts?** Actual TMP117 sensor temperature, clinical wrist-skin temperature, estimated core temperature and fever screening. Only the actual sensor is currently produced; wrist is unverified/null, core is null and fever unavailable.

2. **Why is the actual TMP117 range `-55..150` while wrist is `-10..60`?** The actual range is the sensor code range; the wrist range is a clinical guard for a future verified wrist observation. The actual reading is not promoted into the wrist field.

3. **What exact fever message is required? Core temperature unavailable — confirmation required**.

4. **What happens to the wrist fields now?** `wrist_contact_status: "unverified"`, wrist value/raw/timestamp null, wrist status `unverified`. Core null/`not_available`; fever `not_available`.

5. **What evidence would a verified wrist value need?** Verified contact, valid status, usable raw/timestamp, conversion within 0.02 °C, matching actual TMP117 value/raw/timestamp, and wrist in -10..60 °C. No fixed offset.

6. **Why reject 0x0000 and 0xFFFF?** They are canonical stuck-bus patterns, not trustworthy temperatures. Raw 0xFFFF is -0.0078125 °C mathematically, which can look like -0.01 after rounding.

7. **What is the baseline model?** A frozen per-wearer median/MAD reference: center ± max(3 × 1.4826 × MAD, noise floor), after seven days and thirty sampled minutes per day.

8. **Why does the baseline never become fully ready?** Overall readiness requires all four active metrics, including wrist. Unverified wrist values are not stored, so the wrist model cannot qualify. This is intentional; injecting actual TMP117 would be unsafe.

9. **When is a baseline deviation event recorded?** After three consecutive outside readings with no more than 5 s between them, limited to one event per metric per 300 s. It is not a diagnosis.

10. **What are the engine thresholds?** SOS/fall → RED; HR <50 or >120, or SpO2 <90 → YELLOW; both optical values valid with no threshold → GREEN; otherwise UNKNOWN. Actual/wrist temperature do not trigger an invented clinical alert.

11. **Is the engine AI?** No. ai_decision_engine.py is legacy-named deterministic threshold logic. No trained model, weights, classifier or LLM exists.

12. **What does "resolved" mean in the cleanup register?** The listed software behavior is covered by source review and host tests/builds; it does **not** mean physical hardware or clinical use was validated.

13. **What are the open verification items?** TMP117 physical address/identity/config/raw, accuracy, contact method, MAX/MPU behavior, OLED/SOS/end-to-end, clinical/regulatory suitability and CI.

14. **What is the most dangerous false claim?** Presenting an actual sensor or unverified wrist reading as core temperature/fever evidence. The entire policy exists to prevent that.

I.3 Five deeper follow-up questions

1. **"Could you calibrate a wrist-to-core offset from data?"** Not with this project's evidence. A fixed or learned offset would need a validated core reference, controlled conditions, uncertainty analysis and clinical review. Wrist is affected by ambient/contact/perfusion; the

current build intentionally applies none.

2. **"Why freeze the baseline instead of continuously updating it?"** A continuously retrained model can drift toward the condition it is meant to flag and makes comparisons unstable. Freezing gives a stable reference; the trade-off is that a genuine long-term change requires deliberate re-enrollment/analysis.

3. **"Why require three consecutive readings?"** To reduce single-sample transient false events. It is still an engineering heuristic, not a validated clinical rule; false positives/negatives remain possible.

4. **"Why not alert on actual TMP117 temperature?"** No validated clinical threshold or site relationship exists for this package reading. Alerting on it would imply a clinical meaning the project has not established.

5. **"What would make the baseline clinically defensible?"** Reference-grade sensors, validated contact/site, a clinical protocol, representative data, pre-registered analysis, uncertainty/performance evaluation and ethics/regulatory review. None of that exists now.

I.4 Two cross-module questions

1. **"How is your temperature policy enforced outside `temperature_safety.py`?"** Firmware emits the separate/null fields; the bridge rejects contradictions in `_validate_temperature()`/`_validate_fever()`; the store requires verified contact and matching evidence; the dashboard masks and labels actual/wrist/core/fever separately; the deterministic engine reports actual separately and never invents core/fever.

2. **"How does the baseline interact with storage and the dashboard?"** `PersonalBaseline` reads stored minute means through `TelemetryStore`; the dashboard gets summary/model/comparisons/events from `/api/baseline` and `/api/events`. Unverified wrist data is absent from storage, so no wrist model is built and overall readiness remains incomplete.

I.5 Three improvements

1. **Validate and document a wrist-contact method** before any clinical wrist value or wrist baseline is exposed; keep the unverified/null policy until then.

2. **Add a formal evaluation protocol for sensor accuracy, baseline performance and alert false positives/negatives**, including reference instruments, controlled conditions and uncertainty. Do not call the current thresholds clinical.

3. **Add CI and an auditable evidence artifact** that ties tests/builds to the exact source revision, plus hardware-log templates for the open O-01–O-07 items.

I.6 Ten key facts

1. Actual TMP117, clinical wrist, core and fever are four distinct concepts.
2. Actual accepted range: `-55..150 °C`; clinical wrist guard: `-10..60 °C` with verified contact.
3. Current wrist fields are null/unverified; core is null/`not_available`; fever is `not_available`.
4. Exact message: **Core temperature unavailable — confirmation required**.
5. Raw `0x0000`/`0xFFFF` rejected; no fixed offset.
6. Baseline: 7 days, 30 sampled minutes/day, minute means, frozen median/MAD.
7. Noise floors: HR 5 bpm, SpO2 2%, wrist 0.2 °C, motion 0.1 g.
8. Deviation: 3 consecutive outside readings within 5 s, 300 s cooldown.
9. Engine: RED for SOS/fall; YELLOW for HR <50/>120 or SpO2 <90; GREEN only both optical valid; else UNKNOWN. No AI/LLM.
10. 62 Python + 10 Node tests and both builds pass; hardware/clinical validation remains open.

I.7 Five terms

Term	Explanation
Clinical wrist temperature	Peripheral wrist metric requiring verified contact; not core temperature.
Frozen baseline	A per-wearer median/MAD reference created once after qualification and not retrained on every reading.
MAD	Median absolute deviation; robust spread measure used with the `1.4826` scale factor.
Deterministic alert	Rule-based status from explicit thresholds; no trained model.
Evidence classification	Labeling claims as source, test, build, prior hardware observation, unverified hardware or plan.

I.8 Three source files to open first

1. temperature_safety.py — exact semantics and fever policy.
2. personal_baseline.py — qualification, median/MAD model, comparison/events.
3. ai_decision_engine.py — deterministic findings/actions/terminal card.

I.9 Three things never to claim without evidence

1. **"The system estimates core temperature or screens for fever."** It outputs null/not_available and the exact confirmation message; no validated method exists.
2. **"The baseline/alert engine is AI or clinically validated."** It is a deterministic statistical/rule prototype with no clinical study.
3. **"The software tests prove sensor accuracy or hardware safety."** They are host-side synthetic tests; physical/reference/contact validation is still required.

I.10 Responsibility checklist

- ☐ I can explain actual/wrist/core/fever without collapsing them.
- ☐ I can state the exact ranges and the 0.02 °C raw tolerance.
- ☐ I can explain why 0x0000/0xFFFF are rejected and no offset is applied.
- ☐ I can state the exact fever message.
- ☐ I can explain the seven-day/30-minute baseline and median/MAD formula.
- ☐ I can explain model freeze and the three-reading/300 s deviation rule.
- ☐ I can explain why the baseline cannot be fully ready now.
- ☐ I can explain every deterministic engine status/threshold.
- ☐ I can say clearly that there is no trained model or LLM.
- ☐ I can classify evidence as source/test/build/prior HW/unverified/plan.
- ☐ I can state which audit items are closed in software and which remain open.
- ☐ I can state the three "never claim" boundaries without prompting.

J. Honest boundaries

J.1 What this member can defend

- The exact temperature/fever policy and its cross-layer enforcement.
- The signed raw conversion, tolerance, stuck-pattern and range rules.
- Why core/fever are unavailable and why no offset is used.
- The baseline qualification/model/deviation logic and its limits.
- The deterministic engine's thresholds/statuses and its non-AI nature.
- The distinction between software-resolved findings and hardware/clinical open items.
- The fresh test/build counts and their limits.

J.2 What this member must not claim

- Personal authorship of all safety/audit files without team confirmation.
- Clinical validation, diagnostic utility or regulatory readiness.
- Sensor accuracy or wrist-contact verification.
- Core-temperature estimation or fever screening.
- A trained model, classifier, neural network or LLM.
- That a baseline deviation is a diagnosis.
- That all audit items are closed.
- That tests/builds prove hardware, accuracy or safety.
- That SHT40/MLX90614 data exists.

J.3 Safe closing sentence

"The safety and analytics layer is implemented as a deliberately conservative prototype: it separates actual sensor, wrist and core, refuses to invent core/fever, builds a frozen statistical baseline only after seven qualified days, and uses deterministic alert rules. The software tests and builds pass, but accuracy, contact, clinical validity and the assembled device remain unverified."